PRACTICAL PREPARATION

LP-III
Two-day practical guide

Simple code. Clear demonstrations.
Answers you can explain aloud.

A DAA	B MACHINE LEARNING	C BLOCKCHAIN
6 assignments 3 mini-project options	6 assignments 2 mini-project options	Wallet + contracts Survey + voting dApp

Prepared for Mohit Kurhade

Savitribai Phule Pune University
Fourth Year Computer Engineering (2019 course)
410246: Laboratory Practice III

Every listed OR choice is covered. Python for DAA/ML, Solidity for blockchain, and a small HTML/JavaScript voting page.

Contents

LP-III: start here, Mohit 3
A1. Fibonacci numbers and step count 7
A2. Job sequencing with deadlines 9
A3. Fractional knapsack 11
A4. 0/1 knapsack using dynamic programming 13
A5. Binomial coefficient using DP 15
A6. 8-Queens with the first queen fixed 16
A7a. Mini-project: matrix multiplication and threads 18
A7b. Mini-project: merge sort and threaded merge sort 20
A7c. Mini-project: naive matching and Rabin-Karp 22
B1. Uber fare regression 24
B2. Email spam classification: KNN and SVM 26
B3. Bank churn classification with a neural network 28
B4. Gradient descent for y=(x+3)^2 30
B5. Diabetes classification using KNN 31
B6. Sales clustering and elbow method 33
B7a. Mini-project: historical stock-return prediction 35
B7b. Mini-project: Titanic survival classifier 37
Blockchain: common execution procedure 39
C1. Install MetaMask and study transaction spending 41
C2. Create a wallet and transfer test Ether 42
C3. Bank smart contract: deposit, withdraw, balance 43
C4. Student records: structures, array and fallback 45
C5. Survey: blockchain types and practical use cases 47
C6. Mini-project: voting contract and browser dApp 48
Final viva revision 52
Validation and practical limits 54
References and scope 55

LP-III: start here, Mohit

This pack covers every experiment and every OR mini-project shown in your three syllabus images. Python is used for DAA and ML. Solidity is used for smart contracts; the voting dApp adds a small HTML/JavaScript page. The complete PDF contains the authored source code, steps, explanations and viva answers.

The screenshots say: Group A - any 4 assignments and 1 mini-project; Group B - any 4 assignments and 1 mini-project; Group C - any 4 assignments and a mini-project. Follow your actual journal and your teacher's allotted practical. Use the two-day plan below to rehearse your recorded experiments first.

1. Install once on Windows

Install Python 3.11 or 3.12 if the lab does not already have a working Python environment. These are conservative choices for the ML libraries. In VS Code, open the extracted LP3_Practical_Pack folder, then open Terminal. The terminal must be in the folder containing requirements.txt.

```
py -3.12 -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe -c "import pandas, sklearn, matplotlib; print('Ready')"
```

If Python 3.12 is not installed but `py --version` shows your working version, use `py -m venv .venv`. DAA uses only Python's standard library. For every command in this guide, replace the first `python` with `.venv\Scripts\python.exe` on Windows. No virtual-environment activation is required.

On Linux:

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
```

Use `.venv/bin/python` instead of `python` in the examples. A Jupyter notebook or Colab can also run ML code, but terminal execution keeps all scripts consistent. In a notebook, remove the command-line path line and explicitly set `path` to your uploaded CSV.

2. Run your first program

```
python group_a/a1_fibonacci.py
```

Enter 7. Expected: F(7) = 13, 7 iterative loop steps and 41 recursive function calls. The two step counts measure different operations, so state the definition before comparing them.

3. Understand the folders

Folder/file	What it contains
group_a/	9 DAA scripts: assignments 1-6 and all 3 mini-project choices
group_b/	8 ML scripts: assignments 1-6 and both mini-project choices
group_c/	Bank.sol, StudentData.sol, Voting.sol and voting_dapp/
datasets/	Put the real college/Kaggle CSVs here
demo_datasets/	Artificial practice CSVs, ready to run without downloading data
outputs/	Graphs and result CSVs created when you run ML scripts
LP3_Complete_Practical_Guide.pdf	Full code, execution steps, explanations and viva answers
LP3_Complete_Practical_Guide.md	Searchable/editable text copy of the guide
VALIDATION.md	What was actually checked and the remaining limits

Artificial CSVs are only for learning commands and code flow. Their accuracy scores are not results on your syllabus datasets. For the examination, use your college's data or the linked Kaggle data and report the output you actually obtain.

4. ML datasets: download, extract, place, run

Open a source link, download its ZIP, extract the CSV, and place/rename it as below. Kaggle may ask you to sign in; competition data may require accepting its rules. You do not need an API key for the provided programs because they read local CSVs.

Practical	Local name under datasets/	Target / important columns
Uber	uber.csv	fare_amount; pickup/dropoff latitude/longitude; pickup_datetime; passenger_count
Email spam	emails.csv	Prediction; numeric word counts; Email No. is dropped
Bank churn	Churn_Modelling.csv	Exited; Geography/Gender are categorical; IDs and Surname are dropped
Diabetes	diabetes.csv	Outcome; Glucose, BloodPressure, SkinThickness, Insulin, BMI
Sales	sales_data_sample.csv	QUANTITYORDERED, PRICEEACH, SALES; no target
Stock	stock.csv	ONE company's daily Date and Close columns, years 2000-2020
Titanic	titanic_train.csv	Survived; Pclass, Age, SibSp, Parch, Fare, Sex, Embarked

Dataset sources from your syllabus:

- Uber: https://www.kaggle.com/datasets/yasserh/uber-fares-dataset
- Email: https://www.kaggle.com/datasets/balaka18/email-spam-classification-dataset-csv
- Bank: https://www.kaggle.com/datasets/barelydedicated/bank-customer-churn-modeling
- Diabetes: https://www.kaggle.com/datasets/abdallamahgoub/diabetes
- Sales: https://www.kaggle.com/datasets/kyanyoga/sample-sales-data
- Stock: https://www.kaggle.com/datasets/sagara9595/stock-data
- Titanic: https://www.kaggle.com/competitions/titanic/data

If the stock download contains multiple files, choose one company; do not combine different companies into one time series. If the column is named `Close  Price`, rename that column to `Close`. Ambiguous dates like 03/04/2019 are interpreted as day/month/year by the script; confirm your file's convention. Adjust `dayfirst` if required.

To practise immediately using artificial data:

```
python group_b/b1_uber_regression.py demo_datasets/uber.csv
python group_b/b2_email_spam.py demo_datasets/emails.csv
python group_b/b3_bank_neural_network.py demo_datasets/Churn_Modelling.csv
python group_b/b4_gradient_descent.py
python group_b/b5_diabetes_knn.py demo_datasets/diabetes.csv
python group_b/b6_sales_clustering.py demo_datasets/sales_data_sample.csv
python group_b/b6_sales_clustering.py demo_datasets/sales_data_sample.csv 3
python group_b/b7a_stock_returns.py demo_datasets/stock.csv
python group_b/b7b_titanic.py demo_datasets/titanic_train.csv
```

For sales, open `outputs/sales_elbow.png` after the first run. The second command illustrates k=3; choose the bend shown by your actual dataset, which can differ. The program also prints hierarchical cluster labels for the same k. For real runs replace `demo_datasets` with `datasets`.

5. Two-day preparation plan: about 6 focused hours each day

Session	Time	Measurable outcome
Day 1 - setup	30 min	Python imports work; CSV locations checked; one script executed
Day 1 - DAA core	2 hr	Type and run Fibonacci, both knapsacks and fixed-queen backtracking
Day 1 - remaining DAA	1 hr	Run jobs, binomial and all three mini options; explain one clearly
Day 1 - ML pattern	2 hr	Run gradient descent, diabetes and email; explain split-fit-predict-evaluate
Day 1 - recall	30 min	Explain 4 programs aloud without reading their code
Day 2 - remaining ML	2 hr	Run Uber, bank ANN, sales and both mini options with correct CSV paths
Day 2 - blockchain	2 hr	Deploy Bank and StudentData; demonstrate voting contract and dApp
Day 2 - mock practical	1.5 hr	Do one allotted-style task from each group without copying steps
Day 2 - final review	30 min	Memorize metric formulas, complexities, key Solidity terms; verify files

For each task: read the idea for 5 minutes, type the code once, run it, change one input, and explain why the output changes. Allocate more time to your journal's chosen mini-project. Take short breaks between sessions and avoid spending the final night on new frameworks.

6. The explanation pattern to use with the external

1. Aim: "This program solves/predicts/classifies ..."

2. Input and output: name the data, target and expected output.

3. Method: describe the 3-5 main steps in order.

4. Demonstration: run the program and interpret the output.

5. Evaluation: DAA - time and space; ML - suitable metrics; blockchain - transaction/gas and state change.

6. Limitation: state one real assumption, such as unit-time jobs or one registered address per voter.

For ML say: "I separate features X from target y, split the data, learn preprocessing only on training data, train the model, predict the held-out test rows, and evaluate those predictions." A pipeline keeps imputation, encoding or scaling with the estimator. For stock data use chronological splitting rather than random splitting.

7. Common errors you can fix quickly

Error	Fix
python/py is not recognized	Use the installed Python launcher or add Python to PATH
No module named pandas/sklearn	Install requirements with the SAME Python used to run the program
FileNotFoundError	Run from pack root; check extracted CSV filename/path, not the ZIP name
KeyError: target/column	Print df.columns; check spelling/case/spaces and the selected dataset
UnicodeDecodeError in sales	Keep encoding="latin1", as provided
OneHotEncoder rejects sparse_output	Use scikit-learn >= 1.3 as listed in requirements.txt
ANN ConvergenceWarning	Check scaling; more iterations may help; report the observed result honestly
KNN or SVM is slow	High-dimensional email data is expensive; wait and keep the same test split
Clustering has different IDs	IDs are arbitrary; compare cluster contents, not label number
Remix: transaction reverts	Read the require message, account, Value field and unit
Wrong balance after changing account	showBalance is per msg.sender; select the depositing account
MetaMask: insufficient funds	Use Sepolia test ETH or practise the contract in Remix VM
Voting: no contract at address	Paste the Voting deployment address on the SAME Sepolia network
Voting: missing ethers library	Keep ethers.umd.min.js next to index.html; serve that folder

8. What to have ready before leaving for the lab

- Code pack and actual extracted datasets, using filenames you have tested.
- Your completed journal and the names of your allotted experiments.
- Successful outputs/screenshots as required by your college.
- A test wallet, Sepolia test ETH and saved contract addresses if allowed.
- A full rehearsal of your chosen mini-project; do not depend on a fresh install at exam time.

Never put a wallet recovery phrase or private key in code, screenshots or a journal. These exercises use test assets. The included contracts are teaching examples.

A1. Fibonacci numbers and step count

Aim

Calculate F(n) using iteration and naive recursion; compare clearly defined operation counts.

Say this to the external

I start with F(0)=0 and F(1)=1. The iterative method keeps the previous two values. The recursive method follows F(n)=F(n-1)+F(n-2), which repeats subproblems and therefore takes many more calls.

Understand the code

- a, b = 0, 1 initializes two consecutive Fibonacci numbers. The simultaneous assignment a, b = b, a+b evaluates the right side first.
- range(n) executes the loop n times; steps counts loop-body executions, not every arithmetic or assignment operation.
- The recursive base case returns immediately for n=0 or 1. Other calls return the sum and add the call counts of both branches, plus one for the current call.
- The code limits n to 30 because naive recursion grows quickly. The displayed series is generated separately in one loop.

Perform

Enter 7.

```
python group_a/a1_fibonacci.py
```

Expected output / observation

```
Series: [0, 1, 1, 2, 3, 5, 8, 13]
Iterative: F(7) = 13; loop steps = 7
Recursive: F(7) = 13; function calls = 41
```

Complexity / interpretation

Iterative calculation: O(n) time and O(1) working space. Naive recursion: O(phi^n) time, often bounded by O(2^n), and O(n) call-stack space. Printing/storing the series uses O(n) extra space.

Quick viva

- What is recursion? A function calls itself on a smaller instance until a base case stops it.
- Why is naive recursion slow? The same F(k) is recomputed in many branches. Memoization stores those results.
- Can these step counts be compared directly? They illustrate different growth patterns, but loop bodies and function calls are different units. State the counting convention.

Complete source: group_a/a1_fibonacci.py

```
"""Fibonacci: iteration and recursion, with clearly defined step counts."""


def iterative(n):
    a, b = 0, 1
    steps = 0
    for _ in range(n):
        a, b = b, a + b
        steps += 1              # One loop-body execution = one step
    return a, steps


def recursive(n):
    if n <= 1:
        return n, 1             # Count this function call
    x, calls_x = recursive(n - 1)
    y, calls_y = recursive(n - 2)
    return x + y, 1 + calls_x + calls_y


if __name__ == "__main__":
    n = int(input("Enter n (0 to 30): "))
    if not 0 <= n <= 30:
        raise SystemExit("Use 0 to 30; naive recursion is slow for large n.")
    series = []
    a, b = 0, 1
    for _ in range(n + 1):
        series.append(a)
        a, b = b, a + b
    print("Series:", series)
    value, steps = iterative(n)
    print(f"Iterative: F({n}) = {value}; loop steps = {steps}")
    value, calls = recursive(n)
    print(f"Recursive: F({n}) = {value}; function calls = {calls}")
```

A2. Job sequencing with deadlines

Aim

Maximize profit by scheduling unit-time jobs before their deadlines using a greedy method.

Say this to the external

I sort jobs by decreasing profit. For each job, I put it in the latest free slot before its deadline. This leaves earlier slots available for jobs with tighter deadlines.

Understand the code

- Each tuple is (name, deadline, profit); all jobs take exactly one time unit and there is one machine.
- sorted(..., key=lambda job: job[2], reverse=True) orders by the profit field.
- slots contains None for free positions. Slot index 0 means time interval 0 to 1, so its completion time is 1.
- range(min(deadline,size)-1,-1,-1) searches backward. break prevents scheduling the same job twice.
- At most n slots are needed for n jobs, even if a deadline is much larger.

Perform

The sample jobs are inside the main block. Edit the tuples if given different input.

```
python group_a/a2_job_sequencing.py
```

Expected output / observation

```
Time slots: ['C', 'A', 'E']
Maximum profit: 142
```

Complexity / interpretation

O(n log n + nD) time and O(n + D) space for this sorted-copy version, where D=min(n, maximum deadline). Worst-case time is O(n^2).

Quick viva

- Why the latest free slot? It preserves earlier slots for jobs whose deadlines are earlier.
- Why is this greedy? Each step chooses the remaining highest-profit feasible job.
- Does it handle variable-duration jobs? No. The correctness of this method depends on unit-time jobs.

Complete source: group_a/a2_job_sequencing.py

```
"""Each job takes ONE unit of time; deadlines and profits are positive."""


def schedule_jobs(jobs):
    jobs = sorted(jobs, key=lambda job: job[2], reverse=True)
    size = min(len(jobs), max((job[1] for job in jobs), default=0))
    slots = [None] * size
    total_profit = 0
    for name, deadline, profit in jobs:
        if profit <= 0:
            continue
        for slot in range(min(deadline, size) - 1, -1, -1):
            if slots[slot] is None:
                slots[slot] = name
                total_profit += profit
                break
    return slots, total_profit


if __name__ == "__main__":
    # (job ID, deadline, profit); edit these values for examiner input.
    jobs = [("A", 2, 100), ("B", 1, 19), ("C", 2, 27),
            ("D", 1, 25), ("E", 3, 15)]
    slots, profit = schedule_jobs(jobs)
    print("Time slots:", slots)
    print("Maximum profit:", profit)
```

A3. Fractional knapsack

Aim

Maximize value within a weight capacity when fractions of items are allowed.

Say this to the external

I calculate value per unit weight, sort by that ratio, and take the highest-ratio items first. If only part of the next item fits, I take that fraction.

Understand the code

- Each tuple is (name, weight, value). Weight must be positive and capacity non-negative.
- item[2] / item[1] is value divided by weight, not total value alone.
- taken_weight = min(weight, capacity) handles both a whole item and a fraction.
- fraction * value adds proportional value; capacity decreases by the weight actually taken.

Perform

Sample: weights 10,20,30; values 60,100,120; capacity 50.

```
python group_a/a3_fractional_knapsack.py
```

Expected output / observation

```
Item A: take 1.00 of the item
Item B: take 1.00 of the item
Item C: take 0.67 of the item
Maximum value: 240.00
```

Complexity / interpretation

O(n log n) time due to sorting and O(n) space for the sorted copy and selected-item list.

Quick viva

- Why use the ratio? It measures the value earned for each unit of limited capacity.
- Can the same greedy method solve 0/1 knapsack? Not in general: indivisible items can make a locally best ratio globally suboptimal.
- What is 0.67 here? The display rounds 20/30. The calculation uses the actual fraction, not the rounded text.

Complete source: group_a/a3_fractional_knapsack.py

```
"""Greedy knapsack: an item may be divided into fractions."""


def fractional_knapsack(items, capacity):
    # Each item is (name, weight, value); weights must be positive.
    items = sorted(items, key=lambda item: item[2] / item[1], reverse=True)
    total = 0.0
    selected = []
    for name, weight, value in items:
        if capacity == 0:
            break
        taken_weight = min(weight, capacity)
        fraction = taken_weight / weight
        total += fraction * value
        selected.append((name, fraction))
        capacity -= taken_weight
    return total, selected


if __name__ == "__main__":
    items = [("A", 10, 60), ("B", 20, 100), ("C", 30, 120)]
    value, selected = fractional_knapsack(items, 50)
    for name, fraction in selected:
        print(f"Item {name}: take {fraction:.2f} of the item")
    print(f"Maximum value: {value:.2f}")
```

A4. 0/1 knapsack using dynamic programming

Aim

Find maximum value when each item is either fully selected once or excluded.

Say this to the external

dp[i][w] stores the best value using the first i items and capacity w. For each item I compare excluding it with including it, if its weight fits. I then trace the table backward to show the selected items.

Understand the code

- The table has n+1 rows and capacity+1 columns. Row zero represents no items, with value zero.
- Excluding item i uses dp[i-1][w]. Including it uses value[i-1]+dp[i-1][w-weight[i-1]].
- Using the previous row in both cases prevents taking the same item repeatedly.
- During traceback, a changed value means the current item was included; reduce the remaining capacity and continue.

Perform

Sample: weights [10,20,30], values [60,100,120], capacity 50.

```
python group_a/a4_zero_one_knapsack.py
```

Expected output / observation

```
Maximum value: 220
Selected item numbers: [2, 3]
```

Complexity / interpretation

O(nW) time and O(nW) space, where W is the integer capacity. This is pseudo-polynomial in W, not polynomial in the number of bits needed to encode W.

Quick viva

- Why not 240 like fractional knapsack? We cannot take only part of the 30-weight item; the best whole-item choice is items 2 and 3.
- What makes this dynamic programming? Subproblems overlap, and an optimal solution is built from optimal smaller subproblems.
- Why use i-1 in the include case? The item is allowed only once. Reusing the current row could allow repeated selection.

Complete source: group_a/a4_zero_one_knapsack.py

```
"""0/1 knapsack using a two-dimensional dynamic-programming table."""


def knapsack(weights, values, capacity):
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(capacity + 1):
            dp[i][w] = dp[i - 1][w]       # Exclude item i
            if weights[i - 1] <= w:
                include = values[i - 1] + dp[i - 1][w - weights[i - 1]]
                dp[i][w] = max(dp[i][w], include)

    selected = []
    w = capacity
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i - 1][w]:
            selected.append(i)            # Human-readable item numbers
            w -= weights[i - 1]
    return dp[n][capacity], list(reversed(selected))


if __name__ == "__main__":
    weights = [10, 20, 30]
    values = [60, 100, 120]
    value, selected = knapsack(weights, values, 50)
    print("Maximum value:", value)
    print("Selected item numbers:", selected)
```

A5. Binomial coefficient using DP

Aim

Compute C(n,r), the number of ways to choose r objects from n without considering order.

Say this to the external

I use Pascal's recurrence C(n,r)=C(n-1,r-1)+C(n-1,r). The boundary values C(n,0) and C(n,n) are one. A table stores each smaller result once.

Understand the code

- The recurrence separates selections containing the current object from those excluding it.
- j only runs up to min(i,r), because selecting more than i objects from i is impossible.
- Zero-initialized cells outside the valid triangle stay zero. The function returns zero when r is outside 0..n.
- This avoids repeated recursive calculations and does not require factorials.

Perform

Enter n=5 and r=2. Also try r=0 or r=n.

```
python group_a/a5_binomial.py
```

Expected output / observation

```
C(5, 2) = 10
```

Complexity / interpretation

O(nr) time and O(nr) space. A one-dimensional Pascal row could reduce storage to O(r), but the full table is easier to explain.

Quick viva

- What is the factorial formula? C(n,r)=n!/[r!(n-r)!].
- What is the difference between combinations and permutations? Combinations ignore order; permutations consider order.
- Why is C(n,0)=1? There is exactly one way to choose nothing: the empty selection.

Complete source: group_a/a5_binomial.py

```
"""Binomial coefficient using Pascal's recurrence and dynamic programming."""


def binomial(n, r):
    if r < 0 or r > n:
        return 0
    dp = [[0] * (r + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        for j in range(min(i, r) + 1):
            if j == 0 or j == i:
                dp[i][j] = 1
            else:
                dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j]
    return dp[n][r]


if __name__ == "__main__":
    n = int(input("Enter n: "))
    r = int(input("Enter r: "))
    if n < 0:
        raise SystemExit("n must be non-negative.")
    print(f"C({n}, {r}) = {binomial(n, r)}")
```

A6. 8-Queens with the first queen fixed

Aim

Keep an already placed queen fixed and place the remaining seven queens without attacks.

Say this to the external

I store one column for each row. Before placing a queen I check its column and both diagonals against all queens already stored, including the fixed queen. If a later row has no legal move, I remove the last trial queen and try another column.

Understand the code

- board[row]=column; -1 means that row has no queen. One queen per row is built into the representation.
- safe() checks equal columns and abs(row difference)==abs(column difference), which detects both diagonals.
- The initially supplied queen is written before the search, so earlier rows also avoid attacking it.
- place() skips the fixed row. board[row]=-1 undoes a failed trial. Returning True stops at the first full solution.

Perform

For the sample board below enter row=1, column=1. Positions you enter are 1-based; Python uses 0-based indices internally.

```
python group_a/a6_eight_queens.py
```

Expected output / observation

```
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
Columns, using 1-based positions: [1, 5, 8, 6, 3, 7, 2, 4]
```

Complexity / interpretation

Exponential backtracking, commonly described by an O(N!) placement search. In this simple implementation every safety check scans N rows, so do not claim constant-time safety. A conservative overall upper bound is O(N^2 N!). Board and recursion need O(N) space.

Quick viva

- What is backtracking? Try a choice; if it cannot lead to a solution, undo it and explore another choice.
- Why are row checks unnecessary? The recursion handles one row at a time and stores at most one queen in each row.
- Does the code move the first queen? No. It skips that row and checks all other placements against it.

Complete source: group_a/a6_eight_queens.py

```
"""Backtracking with a fixed first queen, entered as a 1-based row/column."""


def solve_queens(n, fixed_row, fixed_col):
    board = [-1] * n               # board[row] = column of the queen
    board[fixed_row] = fixed_col

    def safe(row, col):
        for r, c in enumerate(board):
            if c == -1 or r == row:
                continue
            if c == col or abs(r - row) == abs(c - col):
                return False
        return True

    def place(row):
        if row == n:
            return True
        if row == fixed_row:        # Preserve the initially placed queen
            return place(row + 1)
        for col in range(n):
            if safe(row, col):
                board[row] = col
                if place(row + 1):
                    return True
                board[row] = -1    # Undo and try the next column
        return False

    return board if place(0) else None


if __name__ == "__main__":
    row = int(input("Fixed queen row (1-8): ")) - 1
    col = int(input("Fixed queen column (1-8): ")) - 1
    if not (0 <= row < 8 and 0 <= col < 8):
        raise SystemExit("Row and column must be between 1 and 8.")
    answer = solve_queens(8, row, col)
    if answer is None:
        print("No solution for the given fixed queen.")
    else:
        for c in answer:
            print(" ".join("Q" if j == c else "." for j in range(8)))
        print("Columns, using 1-based positions:", [c + 1 for c in answer])
```

A7a. Mini-project: matrix multiplication and threads

Aim

Compare serial matrix multiplication with one thread per output row.

Say this to the external

Each output cell C[i][j] is the dot product of row i of A and column j of B. In the threaded version, each thread calculates a different complete row. I join all threads before reading the result and compare it with serial output.

Understand the code

- If A is r by k and B is k by c, the output is r by c. The shared inner dimension must match.
- The three loops implement C[i][j] += A[i][k]*B[k][j].
- Threads write different rows, so no two threads update the same output cell. start() begins work; join() waits for completion.
- Benchmarking uses the same matrices and the median of three runs. Timing includes thread creation and joining.

Perform

The script shows 2x2 results and times 20x20, 40x40 and 60x60 matrices.

```
python group_a/a7a_matrix_threads.py
```

Expected output / observation

```
Serial result: [[19, 22], [43, 50]]
Thread result: [[19, 22], [43, 50]]
Timing values vary by machine. Speedup = serial time / threaded time.
```

Complexity / interpretation

Serial work is O(rkc), or O(n^3) for square matrices. Threading does not reduce total arithmetic work. Output memory is O(rc), with O(r) thread-management objects in the threaded version.

Quick viva

- Is threading always faster? No. Normal CPython has the GIL for Python bytecode, and thread creation also costs time. CPU-bound pure-Python loops may be slower.
- Why no lock around a result cell? Each row has a single writer, and we wait for every writer before reading results.
- What is speedup? Serial execution time divided by threaded execution time. A value below one means the threaded version was slower.

Complete source: group_a/a7a_matrix_threads.py

```
"""Serial multiplication versus one Python thread per output row."""
from threading import Thread
from time import perf_counter
from statistics import median
from random import Random


def multiply(a, b, threaded=False):
    rows, inner, cols = len(a), len(b), len(b[0])
    if any(len(row) != inner for row in a):
        raise ValueError("Columns of A must equal rows of B.")
    result = [[0] * cols for _ in range(rows)]

    def calculate_row(i):
        for j in range(cols):
            for k in range(inner):
                result[i][j] += a[i][k] * b[k][j]

    if threaded:
        threads = [Thread(target=calculate_row, args=(i,)) for i in range(rows)]
        for thread in threads:
            thread.start()
        for thread in threads:
            thread.join()          # Wait before reading the result
    else:
        for i in range(rows):
            calculate_row(i)
    return result


def benchmark(a, b, threaded):
    times = []
    for _ in range(3):
        start = perf_counter()
        multiply(a, b, threaded)
        times.append(perf_counter() - start)
    return median(times)


if __name__ == "__main__":
    a = [[1, 2], [3, 4]]
    b = [[5, 6], [7, 8]]
    print("Serial result:", multiply(a, b))
    print("Thread result:", multiply(a, b, True))
    rng = Random(42)
    for n in [20, 40, 60]:
        a = [[rng.randrange(10) for _ in range(n)] for _ in range(n)]
        b = [[rng.randrange(10) for _ in range(n)] for _ in range(n)]
        assert multiply(a, b) == multiply(a, b, True)
        serial = benchmark(a, b, False)
        parallel = benchmark(a, b, True)
        print(f"n={n}: serial={serial:.6f}s threaded={parallel:.6f}s "
              f"speedup={serial / parallel:.2f}")
    print("Thread overhead and the usual CPython GIL can make threads slower.")
```

A7b. Mini-project: merge sort and threaded merge sort

Aim

Compare ordinary merge sort with sorting its two halves in two worker threads.

Say this to the external

Merge sort divides the list into halves, sorts them recursively and merges the sorted halves. The threaded version creates two top-level workers, waits for both, and then performs the final merge.

Understand the code

- merge() moves the smaller current element from the left or right half into the output, then appends leftovers.
- The <= comparison chooses the left element first for ties, preserving stability.
- Only two workers are created. Creating a new thread at every recursive call would produce excessive overhead.
- The script checks correctness against sorted(data) and measures sorted, reverse and random inputs at multiple sizes.

Perform

The list [38,27,43,3,9,82,10] is shown first. Benchmark results follow.

```
python group_a/a7b_merge_threads.py
```

Expected output / observation

```
Sorted: [3, 9, 10, 27, 38, 43, 82]
Times depend on the system; threads are not guaranteed to win.
```

Complexity / interpretation

This implementation takes Theta(n log n) time in both best and worst cases, with O(n) peak auxiliary data storage and O(log n) recursion depth. Slicing creates copies; threading has extra scheduling overhead.

Quick viva

- Is reverse input automatically the worst case? No. Merge sort keeps dividing and merging regardless of input order. Particular merge interleavings affect comparisons, while best/worst growth stays Theta(n log n).
- Which design strategy is used? Divide and conquer: divide into halves, solve each half, combine with merge.
- Why might two threads be slower? The usual CPython GIL limits concurrent Python computation, and thread-management overhead is included.

Complete source: group_a/a7b_merge_threads.py

```
"""Merge sort and a bounded two-worker threaded merge sort."""
from threading import Thread
from time import perf_counter
from statistics import median
from random import Random


def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    return result + left[i:] + right[j:]


def merge_sort(a):
    if len(a) <= 1:
        return a[:]
    mid = len(a) // 2
    return merge(merge_sort(a[:mid]), merge_sort(a[mid:]))


def threaded_sort(a):
    mid = len(a) // 2
    halves = [None, None]

    def sort_half(index, part):
        halves[index] = merge_sort(part)

    t1 = Thread(target=sort_half, args=(0, a[:mid]))
    t2 = Thread(target=sort_half, args=(1, a[mid:]))
    t1.start()
    t2.start()
    t1.join()
    t2.join()
    return merge(halves[0], halves[1])


def benchmark(function, data):
    times = []
    for _ in range(3):
        start = perf_counter()
        answer = function(data)
        times.append(perf_counter() - start)
    assert answer == sorted(data)
    return median(times)


if __name__ == "__main__":
    print("Sorted:", threaded_sort([38, 27, 43, 3, 9, 82, 10]))
    rng = Random(42)
    for n in [1000, 4000, 8000]:
        cases = {"sorted": list(range(n)), "reverse": list(range(n, 0, -1)),
                 "random": [rng.randrange(n) for _ in range(n)]}
        for case, data in cases.items():
            serial = benchmark(merge_sort, data)
            threads = benchmark(threaded_sort, data)
            print(f"n={n}, {case}: serial={serial:.6f}s threads={threads:.6f}s")
    print("Both best and worst asymptotic time are Theta(n log n).")
    print("Sorted/reverse are input cases, not universal best/worst labels.")
```

A7c. Mini-project: naive matching and Rabin-Karp

Aim

Find all occurrences of a pattern in a text using direct comparisons and rolling hashes.

Say this to the external

Naive matching compares the pattern at every possible starting position. Rabin-Karp compares a numeric hash first, updates that hash when the window slides, and checks characters only when the hash matches.

Understand the code

- All reported indices are zero-based. Overlapping matches are included.
- The base and prime define a small polynomial hash. high is base^(m-1) modulo prime.
- The rolling update removes the outgoing character contribution, shifts the remaining hash and adds the incoming character.
- Hash equality is only a candidate match: comparing text[i:i+m] with the pattern avoids false positives caused by collisions.
- An empty pattern is defined to match every boundary; a pattern longer than the text has no matches.

Perform

Text: AABAACAADAABAABA Pattern: AABA

```
python group_a/a7c_string_matching.py
```

Expected output / observation

```
naive positions: [0, 9, 12]
rabin_karp positions: [0, 9, 12]
Each printed time is the average of 100 calls; short-input timing is noisy.
```

Complexity / interpretation

For text length n and pattern length m, naive worst case is O(nm). Rabin-Karp has expected O(n+m) hash work under suitable hashing and few candidate verifications, but worst case O(nm) with collisions or many verified matches. This Python version also creates a size-m slice during verification. Output storage is proportional to the number of matches.

Quick viva

- What is a hash collision? Different strings produce the same hash value.
- Why verify after matching hashes? Equal hashes do not prove equal strings.
- Why use a rolling hash? It updates the next window hash in constant arithmetic work instead of recomputing all m characters.

Complete source: group_a/a7c_string_matching.py

```
"""Naive matching and Rabin-Karp; report all zero-based match positions."""
from time import perf_counter


def naive(text, pattern):
    n, m = len(text), len(pattern)
    matches = []
    for i in range(n - m + 1):
        for j in range(m):
            if text[i + j] != pattern[j]:
                break
        else:
            matches.append(i)
    return matches


def rabin_karp(text, pattern):
    n, m = len(text), len(pattern)
    if m == 0:
        return list(range(n + 1))
    if m > n:
        return []
    base, prime = 256, 101
    high = pow(base, m - 1, prime)
    pattern_hash = window_hash = 0
    for j in range(m):
        pattern_hash = (base * pattern_hash + ord(pattern[j])) % prime
        window_hash = (base * window_hash + ord(text[j])) % prime
    matches = []
    for i in range(n - m + 1):
        if pattern_hash == window_hash and text[i:i + m] == pattern:
            matches.append(i)      # Verify to reject hash collisions
        if i < n - m:
            window_hash = (base * (window_hash - ord(text[i]) * high)
                           + ord(text[i + m])) % prime
    return matches


if __name__ == "__main__":
    text = input("Text: ")
    pattern = input("Pattern: ")
    for method in [naive, rabin_karp]:
        start = perf_counter()
        for _ in range(100):
            positions = method(text, pattern)
        seconds = (perf_counter() - start) / 100
        print(method.__name__, "positions:", positions, f"time={seconds:.8f}s")
    print("Small-input timings are noisy; repeat with a longer text.")
```

B1. Uber fare regression

Aim

Preprocess Uber ride data, identify training fare outliers, check correlations and compare linear and random-forest regression.

Say this to the external

The target is fare_amount. I clean invalid geographic data, derive trip distance and pickup time features, split the data, remove training fare outliers using IQR, then compare both regressors on the same held-out test rows.

Understand the code

- Latitude and longitude are converted to radians for Haversine distance. It estimates direct geographic distance, not the road route. The stated city-ride scope keeps only distances above zero and at most 100 km; this is a chosen domain rule, not a learned outlier threshold.
- The model uses distance_km, passenger_count, hour and dayofweek. We exclude index/key fields that have no useful meaning for prediction.
- IQR=Q3-Q1. Training fares outside Q1-1.5*IQR to Q3+1.5*IQR are counted and removed; the valid test set remains untouched.
- The correlation table is computed on training data. Correlation suggests linear association and does not prove causation.
- The pipeline learns missing-value medians on X_train. predict() applies the same preprocessing to X_test.

Perform

Use the supplied dataset link. For rehearsal replace datasets/ with demo_datasets/.

```
python group_b/b1_uber_regression.py datasets/uber.csv
```

Expected output / observation

```
Training fare outlier count; correlation table; R2 and RMSE for both models.
Graph: outputs/uber_correlation.png
Scores must come from your actual run; neither model is guaranteed to win.
```

Complexity / interpretation

Linear regression fits a linear relationship. A random forest combines many decision trees and can model nonlinear interactions. Training cost depends on rows, features and tree count; report measured fit time only if asked.

Quick viva

- What is RMSE? Square root of the mean squared prediction error; lower is better and its unit matches the fare.
- What does R2 measure? 1 - residual sum of squares / total sum of squares. One is a perfect fit; negative values mean worse than a mean-target predictor.
- Why not remove test outliers using training target thresholds? That changes the evaluation population using observed target values. Here we evaluate every valid held-out ride.

Complete source: group_b/b1_uber_regression.py

```
"""Run from pack root: python group_b/b1_uber_regression.py datasets/uber.csv"""
import sys
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error

path = sys.argv[1] if len(sys.argv) > 1 else "datasets/uber.csv"
df = pd.read_csv(path).drop_duplicates()
geo = ["pickup_latitude", "pickup_longitude",
       "dropoff_latitude", "dropoff_longitude"]
df = df.dropna(subset=geo + ["fare_amount", "pickup_datetime"])
valid = (df["fare_amount"] > 0)
for column in geo:
    limit = 90 if "latitude" in column else 180
    valid &= df[column].between(-limit, limit)
df = df[valid].copy()

# Haversine formula: approximate great-circle distance in kilometres.
lat1, lon1, lat2, lon2 = [np.radians(df[c]) for c in geo]
h = np.sin((lat2 - lat1) / 2) ** 2
h += np.cos(lat1) * np.cos(lat2) * np.sin((lon2 - lon1) / 2) ** 2
df["distance_km"] = 6371 * 2 * np.arcsin(np.sqrt(np.clip(h, 0, 1)))
time = pd.to_datetime(df["pickup_datetime"], errors="coerce", utc=True)
df["hour"] = time.dt.hour
df["dayofweek"] = time.dt.dayofweek
# Stated scope for this city-ride demo: positive trips no longer than 100 km.
df = df[(df["distance_km"] > 0) & (df["distance_km"] <= 100)]
features = ["distance_km", "passenger_count", "hour", "dayofweek"]
X_train, X_test, y_train, y_test = train_test_split(
    df[features], df["fare_amount"], test_size=0.2, random_state=42)

# Learn outlier bounds ONLY from training data; keep valid test rows intact.
q1, q3 = y_train.quantile([0.25, 0.75])
iqr = q3 - q1
keep = y_train.between(q1 - 1.5 * iqr, q3 + 1.5 * iqr)
print("Training fare outliers:", int((~keep).sum()))
X_train, y_train = X_train[keep], y_train[keep]
correlation = X_train.assign(fare_amount=y_train).corr()
print("Training correlations:\n", correlation.round(3))
models = {"Linear Regression": LinearRegression(),
          "Random Forest": RandomForestRegressor(
              n_estimators=100, random_state=42, n_jobs=-1)}
Path("outputs").mkdir(exist_ok=True)
for name, regressor in models.items():
    model = make_pipeline(SimpleImputer(strategy="median"), regressor)
    model.fit(X_train, y_train)
    prediction = model.predict(X_test)
    rmse = np.sqrt(mean_squared_error(y_test, prediction))
    print(f"{name}: R2={r2_score(y_test, prediction):.4f}, RMSE={rmse:.4f}")
plt.imshow(correlation, vmin=-1, vmax=1, cmap="coolwarm")
plt.xticks(range(len(correlation)), correlation.columns, rotation=45, ha="right")
plt.yticks(range(len(correlation)), correlation.columns)
plt.colorbar(label="Pearson correlation")
plt.tight_layout()
plt.savefig("outputs/uber_correlation.png")
print("Saved outputs/uber_correlation.png; compare both models on the same test set.")
```

B2. Email spam classification: KNN and SVM

Aim

Classify emails into not-spam (0) and spam (1) and compare two binary classifiers.

Say this to the external

In this emails.csv, each word-count column is already a numeric feature. I remove the email identifier, split features and Prediction, then train scaled KNN and a linear-kernel SVM on the same split.

Understand the code

- This script expects the syllabus word-count emails.csv, not a raw-text SMS spam file. Raw text would require a text vectorizer.
- drop(columns="Prediction") produces X; y contains only Prediction. The target must never appear in X.
- stratify=y maintains approximately the same class proportions in training and test sets.
- KNN with k=5 uses majority voting among the nearest training examples. SVM learns a separating boundary with a margin.
- The confusion matrix fixes label order to [0,1], and the report includes precision, recall and F1 for each class.

Perform

Use the dataset with Email No., numeric word counts and Prediction. SVM can take longer on thousands of word features.

```
python group_b/b2_email_spam.py datasets/emails.csv
```

Expected output / observation

```
For both KNN and SVM: accuracy, 2x2 confusion matrix and classification report.
Rows = actual classes; columns = predicted classes.
The exact scores depend on dataset and library versions.
```

Complexity / interpretation

KNN stores the training examples; brute-force prediction compares the query to training points, roughly O(nd) per query. SVM training cost depends strongly on the data and number of support vectors.

Quick viva

- Why scale word-count features? Distances and margins can be distorted by features with much larger numeric scales.
- What is the difference between k in KNN and k in K-means? KNN k is the neighbor count; K-means k is the cluster count.
- Why examine precision as well as accuracy? False positives can wrongly mark legitimate mail as spam; accuracy alone hides that error type.

Complete source: group_b/b2_email_spam.py

```
"""For the syllabus emails.csv with word-count columns and Prediction target."""
import sys
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

path = sys.argv[1] if len(sys.argv) > 1 else "datasets/emails.csv"
df = pd.read_csv(path)
df.columns = df.columns.str.strip()
df = df.drop(columns=[c for c in df if c.lower() == "email no."
                      or c.lower().startswith("unnamed")])
df = df.dropna(subset=["Prediction"]).drop_duplicates()
X = df.drop(columns="Prediction").apply(pd.to_numeric)
y = df["Prediction"].astype(int)    # 0 = not spam, 1 = spam
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
models = {"KNN": KNeighborsClassifier(n_neighbors=5),
          "SVM": SVC(kernel="linear")}
for name, classifier in models.items():
    model = make_pipeline(SimpleImputer(strategy="median"),
                          StandardScaler(), classifier)
    model.fit(X_train, y_train)
    prediction = model.predict(X_test)
    print("\n", name)
    print("Accuracy:", accuracy_score(y_test, prediction))
    print("Confusion matrix [[TN, FP], [FN, TP]]:\n",
          confusion_matrix(y_test, prediction, labels=[0, 1]))
    print(classification_report(y_test, prediction, zero_division=0))
```

B3. Bank churn classification with a neural network

Aim

Predict the dataset Exited label using a small artificial neural network and show accuracy plus confusion matrix.

Say this to the external

I remove identity fields, scale numerical features and one-hot encode categories. An MLP with two hidden layers learns a nonlinear classification rule. Early stopping uses an internal validation split to stop when validation performance stops improving.

Understand the code

- Exited is the binary target. CustomerId, RowNumber and Surname are identifiers, so they are removed.
- ColumnTransformer applies a different preprocessing pipeline to numeric and categorical columns.
- OneHotEncoder creates indicator columns without imposing a fake numeric order on countries or gender.
- MLPClassifier(hidden_layer_sizes=(16,8)) has hidden layers of 16 and 8 neurons. ReLU is the hidden activation and Adam is the optimizer.
- Implemented improvements are scaling, proper categorical encoding and validation-based early stopping. Do not claim they improved a numeric score unless you ran a baseline comparison.

Perform

Read the Exited target and the 14-column bank dataset. This is an sklearn neural network, so installing TensorFlow is unnecessary.

```
python group_b/b3_bank_neural_network.py datasets/Churn_Modelling.csv
```

Expected output / observation

```
Test accuracy; confusion matrix; precision/recall/F1; actual number of training iterations.
The data labels are Exited; no dates are supplied to independently verify a six-month horizon.
```

Complexity / interpretation

A neural network learns weights by repeated forward passes and backpropagation. Time depends on epochs, rows and weight count. A higher training score alone does not prove better generalization.

Quick viva

- What is backpropagation? Using the chain rule to compute how the loss changes with each weight, so the optimizer can update weights.
- What does normalization mean here? StandardScaler centers each numeric feature and divides by its training standard deviation. The same learned transform is applied to test data.
- What is overfitting? A model learns training-specific patterns that do not work well on unseen examples.

Complete source: group_b/b3_bank_neural_network.py

```
"""A small ANN using sklearn MLPClassifier; TensorFlow is not required."""
import sys
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

path = sys.argv[1] if len(sys.argv) > 1 else "datasets/Churn_Modelling.csv"
df = pd.read_csv(path).drop_duplicates().dropna(subset=["Exited"])
X = df.drop(columns=["Exited", "RowNumber", "CustomerId", "Surname"],
            errors="ignore")
y = df["Exited"].astype(int)
numeric = X.select_dtypes(include="number").columns.tolist()
categorical = X.select_dtypes(exclude="number").columns.tolist()
preprocess = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"),
                          StandardScaler()), numeric),
    ("cat", make_pipeline(SimpleImputer(strategy="most_frequent"),
                          OneHotEncoder(handle_unknown="ignore",
                                        sparse_output=False)), categorical)
])
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
ann = MLPClassifier(hidden_layer_sizes=(16, 8), activation="relu",
                    solver="adam", max_iter=500, early_stopping=True,
                    random_state=42)
model = make_pipeline(preprocess, ann)
model.fit(X_train, y_train)
prediction = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, prediction))
print("Confusion matrix [[TN, FP], [FN, TP]]:\n",
      confusion_matrix(y_test, prediction, labels=[0, 1]))
print(classification_report(y_test, prediction, zero_division=0))
print("Training iterations:", ann.n_iter_)
print("Improvements used: scaling, categorical encoding, validation early stopping.")
```

B4. Gradient descent for y=(x+3)^2

Aim

Find the minimum of f(x)=(x+3)^2 beginning at x=2 using gradient descent.

Say this to the external

The derivative is 2(x+3). I repeatedly update x to x minus learning rate times the derivative. The steps move downhill until the gradient is close to zero.

Understand the code

- gradient = 2*(x+3) is the derivative, not the original function value.
- x = x-rate*gradient subtracts the gradient. Adding it would move uphill for this minimization.
- rate=0.1 is the learning rate; tolerance=1e-6 controls the stopping condition.
- The iteration limit prevents an endless loop. This quadratic is convex, so its local minimum is also its global minimum.

Perform

Default x=2, learning rate=0.1. Edit the function arguments to experiment.

```
python group_b/b4_gradient_descent.py
```

Expected output / observation

```
Iteration 1: x=1.000000, f(x)=16.000000
Iteration 2: x=0.200000, f(x)=10.240000
Iteration 3: x=-0.440000, f(x)=6.553600
Minimum near x=-3.000000, f(x)=0, updates=73
```

Complexity / interpretation

O(t) time for t updates and O(1) extra memory. For this function, convergence from x=2 requires 0<rate<1; an excessively large rate can oscillate or diverge.

Quick viva

- What is the exact minimum? x=-3 and f(x)=0, obtained by setting 2(x+3)=0.
- What if the learning rate is very small? Convergence is slower.
- Does a zero gradient always mean a minimum? No. In general it can be a maximum or saddle point; here the positive second derivative proves a minimum.

Complete source: group_b/b4_gradient_descent.py

```
"""Minimize f(x) = (x + 3)^2, beginning at x = 2."""


def gradient_descent(x=2.0, rate=0.1, tolerance=1e-6, iterations=1000):
    for step in range(iterations):
        gradient = 2 * (x + 3)
        if abs(gradient) < tolerance:
            return x, step
        x = x - rate * gradient
        if step < 5:
            print(f"Iteration {step + 1}: x={x:.6f}, f(x)={(x + 3)**2:.6f}")
    return x, iterations


if __name__ == "__main__":
    x, steps = gradient_descent()
    print(f"Minimum near x={x:.6f}, f(x)={(x + 3)**2:.10f}, updates={steps}")
```

B5. Diabetes classification using KNN

Aim

Fit KNN to diabetes.csv and compute confusion matrix, accuracy, error rate, precision and recall.

Say this to the external

Outcome is the target. I treat impossible zero measurements as missing, split the data, impute training medians, scale features and predict with five-neighbor majority voting.

Understand the code

- Glucose, BloodPressure, SkinThickness, Insulin and BMI use zero to represent missing measurements in this exercise.
- Do not replace every zero in the dataset: zero pregnancies and Outcome=0 are valid.
- SimpleImputer and StandardScaler are inside the pipeline, so fit() learns their values only from the training subset.
- Precision and recall use Outcome=1 as the positive class. zero_division=0 handles a model that predicts no positive cases.

Perform

The supplied dataset can name its pedigree column Pedigree; the script keeps all non-target numeric features and supports that name.

```
python group_b/b5_diabetes_knn.py datasets/diabetes.csv
```

Expected output / observation

```
[[TN, FP], [FN, TP]] followed by accuracy, error rate, precision and recall.
Use actual outputs from your CSV; this is a classification exercise, not a clinical diagnosis.
```

Complexity / interpretation

With a brute-force neighbor search, prediction is approximately O(nd) per query; storage is O(nd). n is the number of training examples and d is the feature count.

Quick viva

- What is recall? TP/(TP+FN): the fraction of actual positive cases detected.
- What is precision? TP/(TP+FP): the fraction of predicted positive cases that are actually positive.
- How should k be selected? Use cross-validation on training data. Do not choose the k giving the best repeatedly observed test-set accuracy.

Complete source: group_b/b5_diabetes_knn.py

```
"""KNN classification with accuracy, error rate, precision and recall."""
import sys
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             precision_score, recall_score)

path = sys.argv[1] if len(sys.argv) > 1 else "datasets/diabetes.csv"
df = pd.read_csv(path).drop_duplicates().dropna(subset=["Outcome"])
X = df.drop(columns="Outcome").copy()
y = df["Outcome"].astype(int)
# These measurements cannot be zero; pregnancies and Outcome MAY be zero.
missing_zero = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
X[missing_zero] = X[missing_zero].replace(0, np.nan)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
model = make_pipeline(SimpleImputer(strategy="median"), StandardScaler(),
                      KNeighborsClassifier(n_neighbors=5))
model.fit(X_train, y_train)
prediction = model.predict(X_test)
accuracy = accuracy_score(y_test, prediction)
print("Confusion matrix [[TN, FP], [FN, TP]]:\n",
      confusion_matrix(y_test, prediction, labels=[0, 1]))
print(f"Accuracy: {accuracy:.4f}")
print(f"Error rate: {1 - accuracy:.4f}")
print(f"Precision: {precision_score(y_test, prediction, zero_division=0):.4f}")
print(f"Recall: {recall_score(y_test, prediction, zero_division=0):.4f}")
```

B6. Sales clustering and elbow method

Aim

Cluster sales rows using K-means or hierarchical clustering and inspect the elbow plot to choose k.

Say this to the external

There is no target label. I cluster order lines using quantity, unit price and sales amount after scaling. I plot K-means inertia against k and choose a useful bend, then inspect the resulting cluster sizes and feature means.

Understand the code

- encoding="latin1" handles the supplied sales CSV. The chosen numeric columns avoid addresses and other identifiers.
- StandardScaler keeps SALES from dominating simply because its magnitude is larger.
- KMeans alternates assigning points to nearby centroids and updating those centroids. n_init=10 tries several initializations.
- Inertia is the sum of squared distances to assigned centroids. It normally falls as k grows, so the smallest inertia alone is not a useful k rule.
- The optional second run fits both KMeans and AgglomerativeClustering. Hierarchical labels are a separate clustering, not a copy of K-means labels.

Perform

After the FIRST command open outputs/sales_elbow.png. In the SECOND command replace 3 with your chosen elbow k. If no clear elbow exists, state that and justify an exploratory choice.

```
python group_b/b6_sales_clustering.py datasets/sales_data_sample.csv
python group_b/b6_sales_clustering.py datasets/sales_data_sample.csv 3
```

Expected output / observation

```
Inertia table and outputs/sales_elbow.png.
Second run: cluster counts, cluster means and outputs/sales_clusters.csv.
Cluster labels such as 0 or 1 have no fixed business meaning.
```

Complexity / interpretation

A typical K-means fit takes O(n*k*d*t), where t is the iteration count, multiplied by the number of initializations. Agglomerative clustering can require roughly quadratic storage/time or more depending on method; use manageable datasets.

Quick viva

- What is unsupervised learning? Learning structure from input features without known target labels.
- What is a centroid? The coordinate-wise mean of points assigned to a cluster.
- Are these customer segments? Not directly. This code clusters order lines. For customer segmentation first aggregate features by CUSTOMERNAME and then cluster customers.

Complete source: group_b/b6_sales_clustering.py

```
"""First plot the elbow; run again with chosen k. Also shows hierarchical labels."""
import sys
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.cluster import KMeans, AgglomerativeClustering

path = sys.argv[1] if len(sys.argv) > 1 else "datasets/sales_data_sample.csv"
df = pd.read_csv(path, encoding="latin1")
features = ["QUANTITYORDERED", "PRICEEACH", "SALES"]
data = df[features].apply(pd.to_numeric, errors="coerce")
data = SimpleImputer(strategy="median").fit_transform(data)
data = StandardScaler().fit_transform(data)
maximum = min(10, len(data))
ks = list(range(1, maximum + 1))
inertias = []
for k in ks:
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(data)
    inertias.append(model.inertia_)
Path("outputs").mkdir(exist_ok=True)
plt.plot(ks, inertias, "o-")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Inertia (within-cluster squared distance)")
plt.title("Choose k near the bend; no single k is guaranteed")
plt.tight_layout()
plt.savefig("outputs/sales_elbow.png")
print("k and inertia:", list(zip(ks, [round(x, 2) for x in inertias])))
if len(sys.argv) < 3:
    print("Open outputs/sales_elbow.png, then rerun with a chosen k.")
    print("Example: python group_b/b6_sales_clustering.py", path, "3")
    raise SystemExit(0)
k = int(sys.argv[2])
if not 1 <= k <= maximum:
    raise SystemExit(f"Choose k between 1 and {maximum}.")
df["KMeansCluster"] = KMeans(n_clusters=k, random_state=42,
                             n_init=10).fit_predict(data)
df["HierarchicalCluster"] = AgglomerativeClustering(
    n_clusters=k).fit_predict(data)
print("KMeans cluster counts:\n", df["KMeansCluster"].value_counts().sort_index())
print("KMeans means:\n", df.groupby("KMeansCluster")[features].mean().round(2))
df.to_csv("outputs/sales_clusters.csv", index=False)
print("Saved outputs/sales_clusters.csv (cluster IDs are arbitrary labels).")
```

B7a. Mini-project: historical stock-return prediction

Aim

Analyze one Indian stock series from 2000-2020 and predict the next trading observation's return.

Say this to the external

I sort one company's prices by date and calculate daily returns. Features are today's return and three past returns; the target is the next return. I train on the earlier period and test on the later period, then compare the model against predicting zero return.

Understand the code

- pct_change() computes (Close_t-Close_previous)/Close_previous. A return of 0.02 means 2 percent.
- shift(1), shift(2) and shift(3) use older observations; shift(-1) creates the future target, which must not be included as a feature.
- Training and test data are split in time order. One boundary row is excluded because its target reaches into the test period.
- The latest feature row is saved before dropping its unknown future target; this allows an actual next-observation forecast after the series ends.
- The linear model is a simple baseline. Next-price illustration equals latest close times (1+predicted return). Historical prices may need corporate-action adjustment.

Perform

Choose ONE daily company file with Date and Close. The program filters 2000-2020; if your file has other names, rename Date/Close and confirm its date format.

```
python group_b/b7a_stock_returns.py datasets/stock.csv
```

Expected output / observation

```
MAE/RMSE for regression and the zero-return baseline; direction accuracy.
A historical next-observation forecast; outputs/stock_returns.png.
This does not claim reliable future market returns or a forecast for today.
```

Complexity / interpretation

Linear regression with a small fixed number of lag features is inexpensive. The main evaluation issue is time order and avoiding future information, rather than a sophisticated model.

Quick viva

- Why not a random train/test split? It can train on later information while testing earlier periods, making the forecast evaluation unrealistic.
- What is the zero-return baseline? Predict no price change; a model should be compared with this simple reference.
- Is next day necessarily the next calendar day? No. It is the next trading observation in the daily dataset; weekends and holidays may be missing.

Complete source: group_b/b7a_stock_returns.py

```
"""Next-day returns, chronological test set, and a zero-return baseline."""
import sys
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error

path = sys.argv[1] if len(sys.argv) > 1 else "datasets/stock.csv"
df = pd.read_csv(path)
df.columns = df.columns.str.strip()
# Use ONE company's daily CSV with Date and Close columns, in rupees.
df["Date"] = pd.to_datetime(df["Date"], format="mixed", dayfirst=True, errors="coerce")
df["Close"] = pd.to_numeric(df["Close"].astype(str).str.replace(",", ""),
                            errors="coerce")
df = df.dropna(subset=["Date", "Close"])
df = df[df["Close"] > 0].sort_values("Date").drop_duplicates("Date")
df = df[df["Date"].between("2000-01-01", "2020-12-31")].copy()
df["return"] = df["Close"].pct_change()
features = ["return"]
for lag in [1, 2, 3]:
    column = f"lag_{lag}"
    df[column] = df["return"].shift(lag)
    features.append(column)
df["target"] = df["return"].shift(-1)
# Save the most recent usable features BEFORE dropping the unknown target.
latest = df.dropna(subset=features).tail(1)
data = df.dropna(subset=features + ["target"]).copy()
split = int(len(data) * 0.8)
if split < 5 or len(data) - split < 2:
    raise SystemExit("Need at least 10 usable dated rows from 2000-2020.")
# Exclude the boundary row: its next-day target reaches the test period.
train, test = data.iloc[:split - 1], data.iloc[split:]
model = LinearRegression().fit(train[features], train["target"])
prediction = model.predict(test[features])
baseline = np.zeros(len(test))
for name, values in [("Linear regression", prediction), ("Zero-return baseline", baseline)]:
    print(name, "MAE:", mean_absolute_error(test["target"], values),
          "RMSE:", np.sqrt(mean_squared_error(test["target"], values)))
print("Direction accuracy:", np.mean((prediction > 0) == (test["target"].to_numpy() > 0)))
forecast = model.predict(latest[features])[0]
print("Next-day predicted return after", latest["Date"].iloc[0].date(), ":", forecast)
print("Illustrative predicted price:", latest["Close"].iloc[0] * (1 + forecast))
Path("outputs").mkdir(exist_ok=True)
plt.plot(test["Date"], test["target"], label="Actual next-day return", alpha=0.7)
plt.plot(test["Date"], prediction, label="Predicted next-day return", alpha=0.7)
plt.legend()
plt.xticks(rotation=30)
plt.tight_layout()
plt.savefig("outputs/stock_returns.png")
print("This is an educational historical forecast, not a reliable market prediction.")
```

B7b. Mini-project: Titanic survival classifier

Aim

Predict Survived using passenger class, age, family counts, fare, sex and embarkation location.

Say this to the external

I use the labelled training CSV, fill missing numeric and categorical values using training data, encode categories, and train a small-depth random forest. I evaluate on a held-out portion before optionally predicting the separate Kaggle test file.

Understand the code

- Survived is the target. We choose useful features rather than feeding names or passenger IDs directly into the model.
- The numeric pipeline imputes medians. The categorical pipeline imputes the most frequent category and one-hot encodes it.
- max_depth=5 limits tree complexity; random_state fixes the sampling seed.
- Kaggle test.csv has no Survived labels. You cannot calculate local test accuracy on that file without its ground truth.
- When the optional external CSV is supplied, the model is retrained on all labelled rows only after holdout evaluation.

Perform

The second command is optional; rename the actual Kaggle train.csv and test.csv as shown to avoid confusing them with the held-out split.

```
python group_b/b7b_titanic.py datasets/titanic_train.csv
python group_b/b7b_titanic.py datasets/titanic_train.csv datasets/titanic_test.csv
```

Expected output / observation

```
Holdout accuracy, confusion matrix and classification report.
With the optional external file: outputs/titanic_predictions.csv.
```

Complexity / interpretation

A random forest trains multiple decision trees and combines their votes. Depth limits reduce the size of each tree; the script uses 100 trees with maximum depth 5.

Quick viva

- What is a random forest? An ensemble of decision trees fitted using random samples/features, with classification decided by combined votes.
- How is the holdout different from Kaggle test.csv? The holdout is taken from labelled train.csv and can be scored locally; the separate test.csv lacks Survived.
- Why handle unknown categories? An unseen category in later data should not cause the encoder to fail.

Complete source: group_b/b7b_titanic.py

```
"""Train on Kaggle train.csv, holding out 20 percent for evaluation."""
import sys
from pathlib import Path
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

path = sys.argv[1] if len(sys.argv) > 1 else "datasets/titanic_train.csv"
df = pd.read_csv(path).dropna(subset=["Survived"])
numeric = ["Pclass", "Age", "SibSp", "Parch", "Fare"]
categorical = ["Sex", "Embarked"]
X, y = df[numeric + categorical], df["Survived"].astype(int)
preprocess = ColumnTransformer([
    ("num", SimpleImputer(strategy="median"), numeric),
    ("cat", make_pipeline(SimpleImputer(strategy="most_frequent"),
                          OneHotEncoder(handle_unknown="ignore")), categorical)
])
model = make_pipeline(preprocess, RandomForestClassifier(
    n_estimators=100, max_depth=5, random_state=42))
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
model.fit(X_train, y_train)
prediction = model.predict(X_test)
print("Holdout accuracy:", accuracy_score(y_test, prediction))
print("Confusion matrix [[TN, FP], [FN, TP]]:\n",
      confusion_matrix(y_test, prediction, labels=[0, 1]))
print(classification_report(y_test, prediction, zero_division=0))
if len(sys.argv) > 2:
    # Optional Kaggle test.csv has NO Survived column.
    external = pd.read_csv(sys.argv[2])
    model.fit(X, y)               # Retrain on all labelled rows after evaluation
    output = pd.DataFrame({"PassengerId": external["PassengerId"],
                           "Survived": model.predict(external[numeric + categorical])})
    Path("outputs").mkdir(exist_ok=True)
    output.to_csv("outputs/titanic_predictions.csv", index=False)
    print("Saved outputs/titanic_predictions.csv")
```

Blockchain: common execution procedure

Use Solidity in Remix, not Python. Open https://remix.ethereum.org. Keep your `.sol` files from group_c/ as local backups. Practise with Remix VM first; for the required public test-network demonstration redeploy using MetaMask on Sepolia. The browser dApp needs a wallet-accessible chain, so use Sepolia for the provided page.

1. Create/open a workspace, create the chosen .sol file, and paste its full source.

2. Open Solidity Compiler. Select compiler 0.8.30 (compatible with pragma ^0.8.20) and choose the Shanghai EVM target for consistency with the local validation. Compile.

3. Open Deploy & Run Transactions. For rehearsal select a Remix VM environment; its accounts have simulated funds.

4. Select the correct contract name. Leave Value at zero for deployment because these constructors are nonpayable. Click Deploy.

5. Expand the instance under Deployed Contracts. Read methods return values; write methods create transactions and change state.

6. For a public testnet run, select Sepolia in MetaMask, then choose Browser Extension in current Remix. Some older lab interfaces label this Injected Provider - MetaMask. Connect the test account and redeploy.

7. Expand each successful transaction in the terminal/explorer and record the actual transaction hash, gas used and effective gas price.

Remix VM balances and transactions are simulated. They are not public Sepolia transactions, and a contract deployed in a standalone Remix VM is not directly reachable by the MetaMask page. Keep that distinction clear in your demonstration.

Solidity terms to explain

Term	Simple explanation
contract	Program with state stored and executed on the blockchain
pragma	Compatible compiler-version requirement
uint256	Non-negative 256-bit integer; default checked arithmetic in Solidity 0.8
address	Ethereum account or contract identifier
msg.sender	Immediate caller of the current function
msg.value	Wei attached to the current call
payable	Function may receive Ether, or address may be used to send Ether
require	Validate a condition; if false the call reverts
mapping	Key-to-value storage, similar to a dictionary
struct	A record combining related fields
storage	Persistent contract data
memory	Temporary in-call data
view	Reads state but does not modify it
pure	Does not read or modify contract state
fallback	Handles calls with no matching function selector
ABI	Interface describing how to encode function calls and decode outputs

Calling a view function from the UI through an off-chain read does not create a paid transaction. The same read still consumes execution gas if another contract performs it inside an on-chain transaction.

C1. Install MetaMask and study transaction spending

Aim: install the browser wallet and explain how an Ether transfer spends amount plus transaction fee. This is a procedure; no separate program is required.

1. Open https://metamask.io and use its official download route for your browser. Install the extension and create a test wallet if you do not already have one.

2. Secure its recovery information privately. Do not put it in a journal, screenshot or program. Public account addresses can be shown for the demonstration.

3. In the network menu enable Show test networks and select Sepolia. Its chain ID is 11155111.

4. Obtain Sepolia test ETH from a faucet listed through the Ethereum network documentation, or your lab's funded test account. Faucet availability and requirements can change; prepare this before the exam.

5. Copy a second test account address, send a small amount such as 0.001 Sepolia ETH, inspect the fee estimate, confirm, and wait for the receipt.

6. Record amount, sender, receiver, transaction hash, gas used, effective gas price and actual fee. Verify on the Sepolia explorer linked by the wallet.

Actual execution fee = gas used * effective gas price. If the price is in gwei, multiply by 10^-9 to express it in ETH. Gas limit is a maximum, not necessarily the amount consumed. The sender's balance decreases by transferred value plus fee.

Illustration only: a simple transfer using 21000 gas at an effective 20 gwei costs 0.00042 ETH. Sending 0.001 ETH would reduce the sender by 0.00142 ETH. Record the real values from your receipt, not these example numbers.

Explain to the external: "MetaMask manages keys and signs transactions; it does not store the coins inside the browser. The blockchain records balances. A transaction fee pays for execution, while the transferred amount goes to the recipient."

Viva: gas is a unit of execution work; gas price is the cost per unit. A reverted mined transaction can still consume gas. A wallet address is public, while a private key/recovery phrase controls access.

C2. Create a wallet and transfer test Ether

Aim: create/use a wallet account and perform a testnet transaction. No separate code is needed.

1. In MetaMask create a new test wallet or add another account in an existing test wallet. Adding an account is different from creating a new independent recovery phrase.

2. Name two test accounts, for example Sender and Receiver. Copy Receiver's public address and verify the address before sending.

3. Fund Sender with Sepolia test ETH. Receiver does not need gas just to receive a normal transfer.

4. From Sender choose Send, paste Receiver, set a small test amount, review network and fee, confirm, and wait.

5. Switch to Receiver and verify the received balance. Open transaction details and record its hash and actual fee.

6. Repeat a small transfer if needed to compare fee estimates under different network conditions; do not claim that every transfer always costs the same.

Expected observation: Receiver gains the transferred amount; Sender loses that amount plus the transaction fee. In Remix VM a similar transfer uses simulated balances and does not require a faucet, but that does not fulfil a public testnet receipt requirement.

Explain to the external: "A public address identifies the account. The wallet signs with a private key to authorize spending. All these transfers are on Sepolia, so they use test assets."

Viva: a wallet can manage multiple accounts; a smart-contract address contains code, while an ordinary externally owned account is controlled through a signing key. The transaction hash identifies one transaction, not the wallet.

C3. Bank smart contract: deposit, withdraw, balance

Aim

Maintain a deposited Ether balance for each caller on an Ethereum test network.

Say this to the external

The mapping stores each address's balance in wei. deposit adds msg.value to the caller's balance. withdraw checks that balance, reduces it first, and then sends the requested Ether. showBalance returns only the current caller's ledger balance.

Understand the code

- mapping(address=>uint256) links each address to its own balance.
- deposit is payable, so it can receive Ether. The amount comes from Remix Value and msg.value, not an ordinary function argument.
- withdraw takes an amount in wei. The require statements reject zero and insufficient balances.
- The balance is reduced before the external call: checks, effects, interactions. If the transfer fails, require reverts the whole transaction.
- private limits Solidity access, not public-chain visibility. This is a teaching contract, not an audited banking product.

Perform

1. Select account A. Set Value to 1 ether and call deposit().
2. Set Value back to 0, then call showBalance().
3. Call withdraw(200000000000000000), which is 0.2 ETH.
4. Call showBalance() again.
5. Switch to account B; its deposited balance should be zero.
6. From B try withdraw(1); it must revert. Redeploy on Sepolia with a smaller affordable test deposit if needed.

```
Open group_c/Bank.sol in Remix; compile and deploy using the common procedure.
```

Expected output / observation

```
After a 1 ETH deposit: 1000000000000000000 wei.
After a 0.2 ETH withdrawal: 800000000000000000 wei.
A second account starts with zero. Wallet balance additionally reflects transaction fees.
```

Complexity / interpretation

Mapping lookups and these state updates do not iterate over all customers. Gas is still charged for deployment and state-changing transactions. Direct plain Ether transfers are not accepted by this contract: use deposit().

Quick viva

- Why is deposit payable? It receives attached Ether through msg.value.
- Why reduce the balance before sending? An external transfer may call recipient code. Updating the ledger first prevents withdrawing the same credited amount again.
- Why does the wallet balance differ from showBalance? The wallet contains external ETH and pays gas; showBalance is only that address's deposit ledger within this contract.

Complete source: group_c/Bank.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

// Educational ETH bank: each address has its own deposited balance.
contract Bank {
    mapping(address => uint256) private balances;

    function deposit() public payable {
        require(msg.value > 0, "Send some ETH");
        balances[msg.sender] += msg.value;
    }

    function withdraw(uint256 amountWei) public {
        require(amountWei > 0, "Amount must be positive");
        require(balances[msg.sender] >= amountWei, "Insufficient balance");
        balances[msg.sender] -= amountWei; // Effect before external interaction
        (bool success, ) = payable(msg.sender).call{value: amountWei}("");
        require(success, "ETH transfer failed");
    }

    function showBalance() public view returns (uint256) {
        return balances[msg.sender];
    }
}
```

C4. Student records: structures, array and fallback

Aim

Store student records using a struct and dynamic array, trigger fallback, and observe gas.

Say this to the external

A Student struct groups roll number, name and marks. An array stores all records. addStudent changes blockchain state, getStudent reads one index, and the fallback function counts calls that do not match a declared function.

Understand the code

- Student[] is a dynamic array of Student records. push appends one record.
- string memory name is temporary input; the array stores its persistent copy.
- The getter validates the zero-based index before reading the record.
- fallback is external and nonpayable here: send zero Ether when testing it. Its counter update is a state change and therefore uses transaction gas.

Perform

1. Call addStudent with 1,"Mohit",85 and Value=0.

2. Call count(): expect 1. Call getStudent(0).

3. To test fallback, compile/deploy FallbackCaller.sol, then call trigger with the StudentData contract address.

4. Read fallbackCalls(): expect 1. The helper uses unmatched calldata 0x12345678.

5. Expand transaction receipts for deployment, addStudent and trigger; record actual gas used and effective gas price.

6. Repeat addStudent with a different record and compare receipts without assuming a fixed cost.

```
Open group_c/StudentData.sol in Remix; compile and deploy.
```

Expected output / observation

```
count = 1
getStudent(0) = (1, "Mohit", 85)
fallbackCalls = 1 after one helper call.
The helper receipt includes both the outer call and the nested fallback execution.
```

Complexity / interpretation

Array append and indexed access do not search the full list; gas also depends on dynamic string storage. A longer name can change storage work. The off-chain getter call has no paid transaction fee.

Quick viva

- Why does fallback run? The supplied 4-byte selector matches none of the declared functions.
- What is the difference between receive and fallback? receive handles empty-calldata Ether calls when defined; fallback handles unmatched function calls, and can also handle empty calldata when no receive exists. Receiving ETH requires payable.
- Who may add student records here? Any caller. This simple exercise has no access restriction; an owner/role check would be needed for controlled records.

Complete source: group_c/StudentData.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract StudentData {
    struct Student {
        uint256 roll;
        string name;
        uint256 marks;
    }

    Student[] private students;
    uint256 public fallbackCalls;

    function addStudent(uint256 roll, string memory name, uint256 marks) public {
        require(roll > 0 && marks <= 100, "Invalid roll or marks");
        students.push(Student(roll, name, marks));
    }

    function getStudent(uint256 index) public view returns (uint256, string memory, uint256) {
        require(index < students.length, "Invalid index");
        Student storage student = students[index];
        return (student.roll, student.name, student.marks);
    }

    function count() public view returns (uint256) {
        return students.length;
    }

    // Called for an unknown function selector; accepts ZERO ETH only.
    fallback() external {
        fallbackCalls += 1;
    }
}
```

Optional fallback caller source

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

// Optional helper when Remix's low-level interaction input is hard to find.
contract FallbackCaller {
    function trigger(address target) public {
        require(target.code.length > 0, "Target must be a contract");
        (bool success, ) = target.call(hex"12345678");
        require(success, "Fallback call failed");
    }
}
```

C5. Survey: blockchain types and practical use cases

Aim: compare blockchain network types and relate their access model to suitable applications. This is a short survey, so no executable code is required.

Survey report

A blockchain is a replicated ledger where participants agree on an ordered history of transactions using a consensus protocol. Hash links make unauthorized changes evident, while the network's rules determine who can submit transactions and validate updates. A blockchain is useful when several parties need a shared record and do not want one participant to unilaterally control it. It is not automatically the best substitute for an ordinary database.

Type	Who participates/validates?	Example and practical use
Public	Broadly open participation under protocol rules	Bitcoin payments; Ethereum smart-contract applications and token transfers
Private	One organization controls membership and permissions	A permissioned Fabric deployment for internal asset tracking or audit records
Consortium	A group of organizations governs membership	A shared permissioned supply-chain ledger across manufacturer, transporter and retailer
Hybrid	A design combines restricted records with selected public proofs	Confidential documents kept privately while their hashes are published for verification

Public/private describes access, while proof of work, proof of stake and other protocols describe consensus. These are different classifications. Hybrid is a design category rather than one universal standardized protocol. Fabric can support private or consortium governance depending on how it is deployed.

Practical application 1 - supply-chain tracking: participants record dispatch, transport and receipt events. A shared audit trail can help trace an item's history. It still depends on truthful initial data: a blockchain cannot by itself prove that a physical product was genuine.

Practical application 2 - certificate verification: store a document hash or reference and verify that a later document matches. Keep personal details off a public ledger when they are not needed. A matching hash shows document consistency, not that the issuer was honest.

Practical application 3 - payments and tokens: account owners authorize transfers; the network checks balances and transaction rules. Fees and confirmation times depend on the chain and network conditions.

Practical application 4 - shared business audit: known organizations can use permissioned membership and transaction policies to coordinate a ledger. Governance must define who may join, endorse updates and resolve errors.

Benefits include a shared audit trail and rule-based updates. Limitations include privacy, throughput, fees, key loss, governance and incorrect source data. Immutability makes accepted history difficult to alter under the network assumptions; it does not mean every blockchain is immune to all attacks.

Conclusion: choose the access model according to participants, trust, privacy and governance. Use public networks for open verifiability, permissioned networks for controlled collaboration, and a normal database when one trusted administrator is sufficient.

Explain to the external: "The main distinction is who can participate and validate. Public chains are open; private chains are controlled by one organization; consortium chains are jointly governed. Hybrid designs combine private information with selected public verification."

Viva: decentralization distributes control; consensus agrees on ledger state; hashing detects changed data but does not encrypt it. Blockchain voting is not automatically secret, identity-safe or suitable for real elections.

C6. Mini-project: voting contract and browser dApp

Aim

Create a small dApp that registers voter addresses, accepts one vote per registered address, and displays counts.

Say this to the external

The owner registers voters. Each registered address can vote once while voting is open. The contract stores candidate totals, and the HTML page reads them through ethers and requests MetaMask to sign state-changing transactions.

Understand the code

- The constructor sets the deploying account as owner and creates Alice, Bob and Charlie with candidate IDs 0,1,2.
- eligible records registration; hasVoted records use of the address. These are different checks.
- vote validates voting status, eligibility, duplicate voting and candidate ID before changing either state value.
- Only owner may register addresses or close voting. The frontend includes these controls, but the contract enforces permission.
- The public candidate getter returns name and votes. The page uses a human-readable ABI, BrowserProvider, a signer and transaction.wait().

Perform

1. Deploy Voting.sol in Remix VM to practise the contract buttons. Register a VM account and vote once; a second vote must fail.

2. For the browser page deploy a NEW instance on Sepolia through MetaMask. Copy its contract address.

3. From pack root run the server command below.

4. Open http://localhost:8000 in the same browser as MetaMask, paste the Sepolia contract address and click Connect wallet.

5. With owner connected, paste a voter address and click Register voter; confirm in MetaMask. You may register the owner for a one-account demonstration.

6. Switch to the registered voter if different, reconnect, select candidate ID 1 and click Vote. Confirm and wait for the receipt.

7. Refresh results; Bob should have one vote. Try a duplicate vote and an unregistered address: both should fail.

8. Reconnect the owner and close voting. Further votes must fail.

```
Deploy Voting.sol, then serve group_c/voting_dapp/ as described below.
```

Expected output / observation

```
Three candidate totals appear in the page. After one vote for ID 1, Bob has 1 vote.
Status displays registration, hasVoted and whether voting is open.
Use only test ETH; no private key is copied into the page.
```

Complexity / interpretation

vote and registration perform a fixed number of checks/updates. Displaying every candidate is O(c) reads for c candidates. This is a public educational vote: ballots are visible, and registration does not by itself verify real-world identity.

Quick viva

- Is one address the same as one human? No. The owner must verify identity outside the contract before registering addresses. One person can otherwise create many accounts.
- Is this a complete dApp without the HTML? The contract can be exercised in Remix, but the supplied HTML page provides the requested user-facing decentralized application interface.
- Where is the data stored? Candidate counts and eligibility are contract state; the page reads that state through the wallet provider.

Complete source: group_c/Voting.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Voting {
    struct Candidate {
        string name;
        uint256 votes;
    }

    Candidate[] public candidates;
    mapping(address => bool) public hasVoted;
    mapping(address => bool) public eligible;
    address public owner;
    bool public votingOpen = true;

    constructor() {
        owner = msg.sender;
        candidates.push(Candidate("Alice", 0));
        candidates.push(Candidate("Bob", 0));
        candidates.push(Candidate("Charlie", 0));
    }

    function registerVoter(address voter) public {
        require(msg.sender == owner, "Only owner can register");
        require(voter != address(0), "Invalid voter");
        eligible[voter] = true;
    }

    function vote(uint256 candidateId) public {
        require(votingOpen, "Voting is closed");
        require(eligible[msg.sender], "Voter not registered");
        require(!hasVoted[msg.sender], "Already voted");
        require(candidateId < candidates.length, "Invalid candidate");
        hasVoted[msg.sender] = true;
        candidates[candidateId].votes += 1;
    }

    function closeVoting() public {
        require(msg.sender == owner, "Only owner can close");
        votingOpen = false;
    }

    function candidateCount() public view returns (uint256) {
        return candidates.length;
    }
}
```

Start the voting page

Run from the extracted pack root. ethers.umd.min.js version 6.16.0 is already included beside index.html, with its MIT license. The library loads locally; blockchain access still needs internet/RPC connectivity and MetaMask.

```
python -m http.server 8000 --directory group_c/voting_dapp
```

Open http://localhost:8000. Keep the terminal running. If port 8000 is busy, use 8001 in the command and URL. Press Ctrl+C in the server terminal when finished. If you redeploy, paste the NEW contract address. Changing accounts or networks resets the page connection, so reconnect before the next action.

Architecture: HTML page -> ethers BrowserProvider -> MetaMask approval/signing -> Sepolia Voting contract. Read methods use provider calls; register/vote/close submit transactions and wait for confirmation. No backend or Node installation is needed to run this page.

Complete authored HTML/JavaScript source

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>LP3 Voting Practical</title>
  <style>
    body { font: 17px system-ui; max-width: 680px; margin: 40px auto; padding: 16px; }
    button, input { font: inherit; padding: 10px; margin: 5px 0; }
    input { width: 94%; }
    button { cursor: pointer; }
    li { padding: 8px; }
    #status { white-space: pre-wrap; overflow-wrap: anywhere; background: #eef3f7; padding: 12px; }
    small { color: #475569; }
  </style>
</head>
<body>
  <h1>College Voting Demo</h1>
  <p>Deploy Voting.sol on Sepolia, paste its address, and connect MetaMask.</p>
  <input id="address" placeholder="Voting contract address: 0x..." aria-label="Contract address">
  <button id="connect">Connect wallet</button>
  <button id="refresh">Refresh results</button>
  <p id="account">Wallet not connected</p>
  <ol id="results" start="0"></ol>
  <input id="candidate" type="number" min="0" max="2" value="0" aria-label="Candidate ID">
  <button id="vote">Vote for this candidate ID</button>
  <details>
    <summary>Owner controls</summary>
    <input id="voter" placeholder="Voter wallet address: 0x..." aria-label="Voter address">
    <button id="register">Register voter</button>
    <button id="close">Close voting</button>
  </details>
  <p id="status" role="status">Use a test wallet and Sepolia test ETH.</p>
  <small>One vote per registered address. Votes are public; voter identity is checked outside this demo.</small>

  <!-- Keep ethers.umd.min.js beside this HTML for offline library loading. -->
  <script src="ethers.umd.min.js"></script>
  <script>
    const byId = id => document.getElementById(id);
    const ABI = [
      "function candidates(uint256) view returns (string name, uint256 votes)",
      "function candidateCount() view returns (uint256)",
      "function hasVoted(address) view returns (bool)",
      "function eligible(address) view returns (bool)",
      "function votingOpen() view returns (bool)",
      "function vote(uint256)",
      "function registerVoter(address)",
      "function closeVoting()"
    ];
    let contract, signer;
    const status = text => { byId("status").textContent = text; };

    async function refresh() {
      if (!contract) throw Error("Connect the wallet first.");
      byId("results").replaceChildren();
      const count = Number(await contract.candidateCount());
      for (let i = 0; i < count; i++) {
        const [name, votes] = await contract.candidates(i);
        const item = document.createElement("li");
        item.textContent = `${name}: ${votes.toString()} votes`;
        byId("results").append(item);
      }
      const account = await signer.getAddress();
      byId("account").textContent = `Account: ${account}`;
      status(`Voting open: ${await contract.votingOpen()}\n` +
             `Registered: ${await contract.eligible(account)}\n` +
             `Already voted: ${await contract.hasVoted(account)}`);
    }

    async function connect() {
      if (!window.ethers) throw Error("Missing ethers.umd.min.js; follow the setup guide.");
      if (!window.ethereum) throw Error("Install MetaMask in this browser.");
      const address = byId("address").value.trim();
      if (!ethers.isAddress(address)) throw Error("Paste a valid contract address.");
      const provider = new ethers.BrowserProvider(window.ethereum);
      await provider.send("eth_requestAccounts", []);
      const network = await provider.getNetwork();
```

```
      if (network.chainId !== 11155111n) throw Error("Switch MetaMask to Sepolia.");
      if (await provider.getCode(address) === "0x") throw Error("No contract at this address on Sepolia.");
      signer = await provider.getSigner();
      contract = new ethers.Contract(address, ABI, signer);
      await refresh();
    }

    async function transact(action) {
      if (!contract) throw Error("Connect the wallet first.");
      const transaction = await action();
      status(`Submitted: ${transaction.hash}\nWaiting for confirmation...`);
      await transaction.wait();
      await refresh();
    }

    function handle(id, action) {
      byId(id).onclick = async () => {
        const button = byId(id);
        button.disabled = true;
        try { await action(); }
        catch (error) { status(error.reason || error.shortMessage || error.message); }
        finally { button.disabled = false; }
      };
    }
    handle("connect", connect);
    handle("refresh", refresh);
    handle("vote", () => {
      const id = Number(byId("candidate").value);
      if (!Number.isInteger(id) || id < 0 || id > 2) throw Error("Choose candidate ID 0, 1 or 2.");
      return transact(() => contract.vote(id));
    });
    handle("register", () => {
      const voter = byId("voter").value.trim();
      if (!window.ethers || !ethers.isAddress(voter)) throw Error("Enter a valid voter address.");
      return transact(() => contract.registerVoter(voter));
    });
    handle("close", () => transact(() => contract.closeVoting()));
    if (window.ethereum) {
      const reset = () => {
        contract = signer = undefined;
        byId("account").textContent = "Wallet changed; reconnect.";
        byId("results").replaceChildren();
        status("Click Connect wallet after changing account or network.");
      };
      window.ethereum.on("accountsChanged", reset);
      window.ethereum.on("chainChanged", reset);
    }
  </script>
</body>
</html>
```

Final viva revision

DAA distinctions

Method	Main idea	Examples here
Greedy	Choose a locally best feasible step	Jobs, fractional knapsack
Dynamic programming	Store overlapping subproblem results	0/1 knapsack, binomial coefficients
Backtracking	Try, validate, recurse, undo on failure	Fixed-first 8-Queens
Divide and conquer	Divide, solve smaller parts, combine	Merge sort
Hash-based filtering	Use hashes to shortlist comparisons	Rabin-Karp

Fractional knapsack with the sample data gives 240; 0/1 gives 220. The difference is allowed item divisibility. Backtracking is not the same as branch and bound: branch and bound uses objective bounds to prune optimization searches. Assignment A4 permits DP OR branch and bound; this pack chooses DP.

ML metric formulas

The code prints the binary confusion matrix with actual rows and predicted columns ordered [0,1]: first row [TN,FP], second row [FN,TP].

Metric	Formula / interpretation
Accuracy	(TP+TN)/(TP+TN+FP+FN)
Error rate	(FP+FN)/total = 1-accuracy
Precision	TP/(TP+FP): correctness among predicted positives
Recall	TP/(TP+FN): coverage of actual positives
F1	2*precision*recall/(precision+recall)
MSE	Mean of (actual-predicted)^2
RMSE	Square root of MSE; same units as target
MAE	Mean absolute prediction error
R2	1 - sum(error^2)/sum((actual-mean(actual))^2)

Example only: TN=40, FP=5, FN=10, TP=45. Accuracy=85/100=0.85; error rate=0.15; precision=45/50=0.90; recall=45/55=0.8182. Recompute with the matrix from your actual run.

Functions the external may point at

Code	Explain it simply
pd.read_csv(path)	Read a CSV into a labelled data table
df.drop / dropna	Remove selected columns/rows or rows with missing required fields
X, y	Input features and the target to predict
train_test_split	Reserve unseen rows for evaluation
random_state=42	Fix the pseudorandom seed for repeatability; 42 is not a special optimal value
stratify=y	Preserve class proportions approximately in each split
SimpleImputer	Learn a training-data replacement for missing feature values
StandardScaler	Subtract training mean and divide by training standard deviation
OneHotEncoder	Turn a category into indicator columns
make_pipeline	Apply preprocessing and an estimator in a defined order
fit	Learn transformation parameters or model parameters from training data
transform	Apply an already learned preprocessing transformation
predict	Produce outputs using the fitted model
fit_predict	Fit a clustering model and return its cluster assignments
print(...:.4f)	Display a float rounded to four decimal places

Supervised learning uses known targets: regression predicts numeric values, classification predicts classes. Clustering is unsupervised: there is no target label in the sales experiment. A neural network is not a separate learning category from supervised learning; here the MLP performs supervised classification.

A three-minute mock explanation

For a DAA task: state the problem, show the example input, trace one iteration or recurrence, run the code, state time and space, then change one input.

For ML: name X and y, show the target column, explain train/test separation, point to preprocessing, fit/predict, then interpret the confusion matrix or regression metrics. Do not invent a fixed accuracy or tune repeatedly on the test set.

For blockchain: identify the network/account, compile/deploy, perform a valid state change, show one expected rejection, and explain gas plus persistent state. A transaction hash is evidence only for the network where that transaction occurred.

Validation and practical limits

Checked on 31 October 2026. Runtime: Python 3.12.14; numpy 2.3.5; pandas 2.2.3; scikit-learn 1.8.0; matplotlib 3.10.8. These are the versions used for validation, not a requirement to upgrade an already working lab environment.

- All Python files compile. All eight ML scripts executed with the included artificial practice CSVs; gradient descent requires no dataset.
- Algorithms were checked against known outputs and randomized brute-force comparisons where appropriate. 0/1 knapsack and job sequencing were compared with exhaustive small cases. Binomial coefficients were compared with math.comb. String matchers were checked with random texts, empty patterns, overlaps and non-ASCII characters.
- All 64 possible fixed-queen positions were checked for a valid 8-Queens solution preserving the chosen square. Matrix serial/threaded results and merge-sort serial/threaded results were compared for correctness, including non-square compatible matrices and empty sort inputs.
- Uber, email, bank churn, diabetes and sales scripts ran on the actual CSVs downloaded from the dataset links in the supplied syllabus. Their expected column names were confirmed.
- The stock script ran on a single-company daily CSV from the linked Indian stock dataset. Only dates in 2000-2020 are used. The Titanic script was exercised using labelled artificial data; the actual competition CSV still needs to be supplied for your final experiment.
- Bank.sol, StudentData.sol, Voting.sol and FallbackCaller.sol compiled with Solidity 0.8.30 targeting Shanghai. The contracts were deployed on a local simulated EVM; deposit, withdrawal, per-account balances, student storage, fallback, voting permissions, duplicate rejection and voting closure were exercised.
- The actual authored voting HTML/JavaScript was executed in a simulated browser DOM connected to a local EVM. Connect, owner registration, voting, displayed counts, duplicate rejection and closing were checked. This does not replace testing the installed MetaMask extension in your own browser.
- No transaction was sent through your wallet or to public Sepolia. Public testnet availability, wallet connection permissions, faucet funding and actual network fees must be checked in your browser before the practical.

Artificial practice data is not the Kaggle data and must not be presented as real experimental evidence. Exact ML scores and timing measurements can change with data, versions and hardware. Use your own observed results. The PDF contains the same authored source as the code files, excluding the third-party minified ethers library and the optional practice-data generator.

References and scope

The assignment list is transcribed from the three syllabus screenshots supplied in this conversation: SPPU Fourth Year Computer Engineering (2019 course), 410246 Laboratory Practice III. Dataset URLs are preserved from those screenshots. All listed OR mini-project choices are included; the college may require only the selected journal options.

Official documentation checked for execution details:

- Python threading and the GIL: https://docs.python.org/3/library/threading.html
- scikit-learn preprocessing and leakage: https://scikit-learn.org/stable/common_pitfalls.html
- Remix Deploy & Run: https://remix-ide.readthedocs.io/en/latest/run.html
- MetaMask test networks: https://support.metamask.io/configure/networks/how-to-view-testnets-in-metamask/
- Ethereum networks and Sepolia: https://ethereum.org/developers/docs/networks/
- Solidity contracts and fallback: https://docs.soliditylang.org/en/v0.8.30/contracts.html
- Solidity security considerations: https://docs.soliditylang.org/en/v0.8.30/security-considerations.html
- ethers v6 provider/signer/contract interface: https://docs.ethers.org/v6/getting-started/
- Hyperledger Fabric permissioned-network concepts: https://hyperledger-fabric.readthedocs.io/en/latest/whatis.html

The code is designed for a clear college demonstration. ML scores must be generated on the actual provided CSVs; artificial practice data is explicitly labelled. A bank Exited label does not independently establish a future six-month observation window. Stock code predicts a next observation after a historical series, not present-day trading performance. Wallet and Remix menu wording can vary by installed version.